CircleTrackRL:
Benchmarking Continuous-Control Reinforcement Learning on a 3D Circular Trajectory Tracking Task

Hayk Nalchajyan · Liana Zhamkochyan

2026

Abstract. Trajectory tracking is a fundamental challenge in autonomous robotics and aerial vehicle control, demanding precise, adaptive control policies in continuous state-action spaces. This paper presents CircleTrackRL, a compact and reproducible research framework for training and benchmarking deep reinforcement learning agents on a 3D circular trajectory tracking task. We introduce a lightweight Gymnasium-compatible simulation environment that models quadrotor-like dynamics and exposes a dense, shaped reward signal tuned for smooth tracking and low steady-state error. We systematically compare three leading continuous-control algorithms—Twin Delayed Deep Deterministic Policy Gradient (TD3), Soft Actor-Critic (SAC), and Deep Deterministic Policy Gradient (DDPG, realized as a TD3 variant)—under a unified Optuna-driven hyperparameter optimization regime. Across 50 evaluation episodes, TD3 achieves the highest mean cumulative reward (124.6 ± 17.3) and lowest tracking RMSE (0.12 m), followed by SAC (118.2 ± 20.1, 0.15 m) and DDPG-as-TD3 (110.5 ± 25.4, 0.18 m). The codebase provides a reproducible, end-to-end pipeline from hyperparameter search to 3D animated visualization, supporting extension to new algorithms and reference trajectories.

1. Introduction

The problem of autonomous trajectory tracking refers to steering a dynamical system to a desired reference trajectory, and is at the crossroads of control theory and machine learning. Traditionally, classical control techniques like PID regulators and model predictive control (MPC) have been used in practice, but they need accurate models of the system and parameter tuning. The alternative of deep reinforcement learning (RL) is somewhat attractive: The agents learn an effective control policy from interacting with a simulated environment, without knowing about any model.

Circular trajectory tracking is a nice test to assess such agents. Geometrically, the task is simple: keep the agent in a circular orbit at a specific radius with a specific angular velocity, but dynamically it is rich with multiple degrees of freedom to be controlled – forces must be applied to all axes simultaneously. This trajectory is very easy to see steady state tracking error, transient response and robustness to observation noise.

This paper presents CircleTrackRL, an open-source framework based on three design principles: (1) reproducibility: each experiment is seeded and each parameter search is reported; (2) comparability: all algorithms are run in the same environment, and under the same reward function and evaluation protocol; and (3) extensibility: it is easy to add a new algorithm, reward function, or reference trajectory without changing the codebase.

The following are a few of our contributions:

A 3D lightweight tracking system for Gymnasium usage that has a controllable radius, angular velocity and observation noise.

A dense reward formulation that weaves a balance between proximity-to-reference, control smoothness, and angular progress.

A single Optuna hyperparameter search for learning rate, network structure, batch size, and soft update rate for TD3, SAC and DDPG.

Comparisons of all three algorithms for mean cumulative reward and root-mean-square tracking error (RMSE) over 50 episodes of evaluation.

Visualization tooling for generating static comparison plots and 3D animated videos of trajectories.

2. Background and Related Work

2.1 Continuous-Control Reinforcement Learning

Much of the current research in deep RL for continuous action spaces relies on policy gradients and actor-critic architectures.Policy gradient and actor-critic architectures are the basis of modern deep RL for continuous action spaces. A variant of the deterministic policy gradient theorem for high-dimensional state-action spaces is called the Deep Deterministic Policy Gradient algorithm. It saves a replay buffer for off policy learning and target networks for stable gradient estimation.

Twin Delayed DDPG overcomes the overestimation bias of Q-learning by keeping two copies of the critic, selecting the minimum Q value for the target, and delaying policy updates to decrease variability. These changes collectively lead to much more robust training curves for benchmark continuous-control tasks.

Soft Actor-Critic uses a maximum entropy approach, which explicitly prefers stochastic policies with high entropy. The additional regularization is intended to enhance exploration and robustness, and can be competitive when it comes to sample efficiency. SAC's automatic temperature tuning eliminates an extra hyperparameter in manual search.

2.2 Trajectory Tracking with RL

Previous studies have used model free RL for stabilizing the quadrotor, for following a given trajectory while flying a fixed-wing aircraft, and for acrobatics on multi-rotor aircraft. The studies are usually on the high fidelity simulators equipped with detailed aerodynamic models. CircleTrackRL develops a point-mass model, while deliberately avoiding any other complicating factors in the simulator, so as to provide a clean and fast iteration space for control-learning and algorithmic comparisons.

2.3 Hyperparameter Optimization for RL

Choosing the hyperparameters of reinforcement learning is very sensitive. There are automated hyperparameter optimization (HPO) frameworks that use Bayesian optimization or TPE sampling to search through a large space efficiently, like Optuna. CircleTrackRL's core application of rigorous HPO has been shown to be crucial in several recent works to fairly compare algorithms.

3. The CircleTrackRL Environment

3.1 Dynamics and State Space

The environment models a point-mass agent navigating three-dimensional space. At each timestep t, the agent observes a state vector

$$s_t = [p_t, v_t, e_t] \in \mathbb{R}^9$$

where $p_t \in \mathbb{R}^3$ is the Cartesian position, $v_t \in \mathbb{R}^3$ is the velocity, and $e_t \in \mathbb{R}^3$ is the vector error between the agent's current position and the target point on the reference circle. Optional Gaussian observation noise $\varepsilon \sim \varphi(0, \sigma^2 I)$ can be added to simulate sensor imperfections.

The action space is a three-dimensional continuous force vector $a_t \in [-1, 1]^3$, clipped and scaled to a physical thrust bound. Integration follows Euler discretization at a fixed timestep of $\Delta t = 0.02$ s.

3.2 Reference Trajectory

The reference trajectory is a horizontal circle of radius R (default 2.0 m) centered at the origin, parameterized by angular position $\theta(t) = \omega t$ where ω is the angular speed (default 0.5 rad s^{-1}). The target point at time t is:

$$g(t) = (R\cos(\omega t), R\sin(\omega t), 0)$$

The episode length is fixed at 500 steps (10 s of simulated time), starting from a random initial position drawn from a uniform distribution within $[-0.5, 0.5]^3$ m of the first reference point.

3.3 Reward Function

The reward function is designed to encourage proximity to the reference point, penalize excessive control effort, and reward progress along the circle:

$$r_t = -\lambda_1 \|e_t\|^2 - \lambda_2 \|a_t\|^2 + \lambda_3 \Delta\theta_t$$

where $\|e_t\|^2$ is the squared position error, $\|a_t\|^2$ is the squared action norm, $\Delta\theta_t$ measures the angular progress made in this step, and $\lambda_1, \lambda_2, \lambda_3$ are weighting coefficients. This dense formulation avoids sparse reward pathologies and provides a smooth gradient signal throughout the episode.

3.4 Gymnasium Compliance

The environment fully implements the Gymnasium API (step, reset, render, close) and passes the Gymnasium environment checker without warnings. This ensures compatibility with any RL library that targets the Gymnasium interface, including Stable-Baselines3, which is used throughout this work.

4. Methods

4.1 Algorithms

In the project the following 3 algorithms are implemented. All of them are imported from Stable-Baseline3 library.

- TD3 — Twin Delayed DDPG with clipped double-Q learning and delayed policy updates.
- SAC — Soft Actor-Critic with entropy regularization and automatic temperature adjustment.
- DDPG-as-TD3 — a DDPG-style agent (no double-Q, no delay) implemented within the TD3 class by disabling its twin-critic features, providing a direct ablation baseline.

4.2 Hyperparameter Optimization

For each algorithm, the study includes 50 trials using the Tree-structured Parzen Estimator (TPE) sampling approach. During each trial, the agent is trained on 50,000 environmental interactions and evaluated in 10 environments based on the average total reward value. The specified search space is presented in Table 1 below.

Table 1. Hyperparameter Search Space

Hyperparameter	Search Range	Scale
Learning rate (actor/critic)	[1e-5, 1e-2]	log-uniform
Network width (hidden units)	{64, 128, 256, 512}	categorical
Batch size	{64, 128, 256, 512}	categorical
Soft update rate (τ)	[5e-4, 5e-2]	log-uniform
Discount factor (γ)	[0.95, 0.999]	uniform

4.3 Final Training

The hyperparameter tuning that provides the best parameters for each algorithm were used in the training process and all the models were saved. Training includes 1000 random exploration steps before gradient updates begin.

4.4 Evaluation Protocol

For evaluation we choose 500 independent episodes and evaluate it deterministically. After evaluation the model provides the mean, standard deviation of the cumulative episode reward, mean and median tracking RMSE for all timesteps in each episode. For RMSE the following formula is used:

$$RMSE = \sqrt{(1/T \Sigma\square \|e\square\|^2)}$$

5. Results

5.1 Hyperparameter Tuning

Each one of these models was tuned using 50 trials performed using Optuna. In the case of TD3 and SAC, good results started appearing after roughly 30 trials, demonstrating their relative stability. On the other hand, DDPG had more variance, which was expected, considering that this approach uses only one critical model, making it inherently less stable than the TD3 method. All three models showed better results with higher batch sizes, usually 256 or 512, as it helped to improve learning since more training data could be included in updates. Furthermore, small learning rates around 3×10^{-4} and 1×10^{-3} were preferred, as they helped to avoid unstable updates. Finally, in terms of the neural network structure, TD3 and SAC preferred larger models with two hidden layers consisting of 256 neurons, while DDPG chose smaller models with 128 neurons.

5.2 Quantitative Comparison

Table 2 summarizes the evaluation results over 50 episodes for each algorithm using its best-found hyperparameters.

Table 2. Evaluation Results (50 Episodes)

Algorithm	Mean Reward	Std Reward	Mean RMSE (m)
TD3	124.6	±17.3	0.12
SAC	118.2	±20.1	0.15
DDPG-as-TD3	110.5	±25.4	0.18

Both measures prove TD3 to be more effective. In terms of RMSE (0.12 m vs 0.15 m for SAC and 0.18 m for DDPG), it is clear that the implementation of the clipping technique along with delayed updates results in improved trajectory tracking. SAC can be regarded as a close second-place performer, although its larger reward variance suggests that it might explore different trajectories because of its stochasticity. As regards the maximum reward variance, it is achieved by DDPG-as-TD3 (±25.4).

5.3 Training Curves

As the reward over time became higher and higher, it can be concluded that all the 3 algorithms managed to improve during the training. TD3 became stable after about 350000 steps, while DDPG was less stable and demonstrates the main drawback of it: it overestimated the values and caused instability. As a proof we see that suddenly it drops the reward. Compared to DDPG, SAC kept improving faster but could not perform as well as TD3. This means that if it had more timesteps, the result would be better.

5.4 Trajectory Quality

Based on the 3D animation of the baseline trajectory and the trajectories for each of the algorithms it can be stated that the difference between the algorithms are evident. The TD3 algorithm , which provided

the highest performance among the chosen approaches, follows the baseline trajectory in a tight and very consistent orbit. The deviation from the original orbit never exceeded 0.15m . About DDPG can be said the following: high RMSE and reward variance, it could not regularly follow the baseline trajectory, which is a result of wobbling and loss of the track. However, it managed to return to the right place and continue the smooth movement. SAC could produce the smooth trajectory, however its trajectory was a bit shifted from the ideal trajectory.

6. Implementation Details

6.1 Software Stack

CircleTrackRL is implemented in Python 3.10+. The main libraries for this project are Gymnasium, which is responsible for the environment API, Optuna package for providing the Bayesian optimization and hyperparameter tuning and Stable-Baseline3 for the chosen reinforcement learning algorithms implementation. The supporter libraries are NumPy for numerical operations, Matplotlib for visualization and animation tasks and Pandas for result storage. The whole project workflow, including the hyperparameter tuning, model training, result storage and visualization is organized in ru_pipeline.sh executable file.

6.2 Repository Structure

- envs/circle_tracking_env.py — Gymnasium environment implementation
- tuning/tune_circle_optuna.py — Optuna objective and study runner
- training/train_best_circle_model.py — final training, evaluation, and CSV logging
- visualization/plot_optimized_results.py — bar/box comparison plots
- visualization/animate_trajectory.py — 3D trajectory animation
- tuning_results/ — Optuna SQLite databases and best-param JSON files
- results/ — evaluation CSV summaries and per-episode traces

6.3 Reproducibility

All the necessary random seeds are global for the whole project and are set before the environment creation. The Bayesian optimization that is executed using Optuna library saves all the necessary information in SQLite database, which allows it to resume the stopped program and searches without any loss of information from previous operations. The necessary dependencies and their appropriate versions are stated in the requirements.txt file for ensuring the successful and up-to-date virtual environment creation.

7. Discussion

7.1 Algorithm Selection Guidance

In order to develop and train a working continuous-control agent on a similar trajectory tracking task, The following 3 algorithms are chosen for the further development and training. The first and foremost choice is TD3, because it is the safest approach for achieving the best tracking accuracy and provides low variance among all algorithms when we provide the same parameters and computational power. Another chosen training algorithm is DDPG as it is the improved version of TD3 and can provide a stronger performance in the scope of this problem, however, it requires more computational power. The third option for training is the SAC algorithm, as during the long training process or when we should reach the maximal robustness in a noisy and stochastic environment.

7.2 Reward Engineering Insights

From the reward engineering can be stated that enough useful feedback was received when the agent received dense rewards. That stimulated the understanding of the learning process and its success. Early experiments with sparse rewards (only penalizing deviation beyond a threshold) failed to converge for all three algorithms within the 500,000-step budget. The angular progress term ($\lambda_3 \Delta\theta$□) was particularly important: without it, agents learned to hover near the starting point of the circle rather than orbit continuously.

7.3 Limitations

The project has a couple of limitations. The first and most obvious limitation is the amount of evaluation episodes, as only 50 episodes are not able to provide a strong feedback about the performance, as the intervals will narrow with additional episodes. Another strong limitation can be considered the fact that we simply cannot take and apply it in real life, as several important phenomena like the aerodynamic drag, turbulence, body orientation are not considered in the training process. As a result, Circle tracker is a good tool for considering the trajectory tracking, but still not ready for the real life testing in nature, where a lot of factors influence the agent.

8. Future Work

As a further development for this project can be found several important directions for improvement and development.

- The first one is providing more generalization and robustness with adding new reference trajectories to the benchmark. It means including more complex curves into the training process for making the model more independent, strong and close to real-life problems.
- Take into consideration other factors that make the circle tracker closer to real life like adding aerodynamic drag or gravitational factors for more realistic simulation.
- Integrate automated CI tests into the model to see the dynamics of the environment and understand the trend whether the algorithm converges or not.
- Train other RL algorithms into the training process like PPO for having a wide range of algorithms and see how different approaches manage to handle the same problem in the same conditions.

9. Conclusion

To sum up, after optimization and model training it can be stated that CircleTrackRL managed to develop and provide a working, compact and reproducible benchmark for continuous-control reinforcement learning on 3D trajectory tracking. All the processes executed during the whole pipeline we can really see the main differences between these 3 models and compare their performances. Based on the results provided after the training process can be stated the following observations:
TD3 has the strongest and best performance in this environment and this task. It reached the highest cumulative reward among 3 models and a solid mean tracking RMSE of 0.12m. The other 2 algorithms were close to the performance of TD3, however, did not manage to show better or equivalent results.